

QMAPS

Mathematical and Technical Project Report

A judge-readable formulation of the route planning system, optimization methods, traffic model, and benchmark evidence

Prepared from the current QMaps repository, its saved benchmark CSV files, statistical outputs, and the implementation inspected on 28 September 2026.

Scope: the mathematical core, implementation choices, measured results, interpretation limits, and a reproducible path for stronger evaluation.

This report distinguishes implemented behavior from research conventions and future validation needs.

Contents

- 1 Executive summary
- 2 Project purpose and system boundary
- 3 Vehicle routing with time windows
- 4 Random keys and Prins split
- 5 Optimization algorithms
- 6 Dynamic traffic and realistic ETA
- 7 Software architecture and operating flow
- 8 Benchmark design and reproducibility
- 9 Results from saved experiments
- 10 Why QPSO underperforms on larger instances
- 11 Evidence limits and recommended next experiments
- 12 Glossary
- 13 References
- Appendix A Current implementation parameters
- Appendix B Mathematical symbols

1 Executive summary

QMaps is a web-based logistics demonstration that accepts a depot, delivery stops, vehicle capacities, time windows, priorities, and traffic adjustments. A FastAPI service builds a route-planning instance and runs a selected optimizer. The frontend displays route assignments, street geometry, stop schedules, traffic annotations, and driver-facing directions.

The optimization core represents a delivery plan as a permutation of customers. Random-key vectors map continuous optimizer positions to permutations; a dynamic-programming split then divides that order into capacity-feasible vehicle routes and prefers a time-window-feasible split when one exists. The benchmark objective is total Euclidean route distance in the instance coordinate scale. In the live Navi Mumbai workflow, the backend also creates travel-time estimates and can include a soft high-priority-arrival term.

The repository contains six named methods: A*, PSO, QPSO, ACO, QACO, and the QA-QPSO hybrid. The hybrid spends an initial portion of its budget on QACO sequence construction, then warm-starts QPSO around that solution. “Quantum-inspired” means classical computation using probability-amplitude or QPSO-style update equations. The application does not run on quantum hardware.

The stored full suite contains 30 runs per algorithm on three selected instance sizes (25, 50, and 100 customers), with 540 rows in each of two historical CSV snapshots. In the newer historical snapshot, the hybrid has the lowest mean route-distance cost on the 25- and 50-customer cases, while ACO has the lowest mean on the 100-customer C101 case. These values are not directly comparable to standard Solomon best-known solutions because this project minimizes distance without the benchmark’s lexicographic minimize-vehicles-first objective, and the smaller RC cases are prefixes of larger files.

The saved historical PSO/QPSO comparison shows no reliable difference on RC101, and PSO has lower cost on RC201 and C101. A separate paired 30-seed rerun using the corrected canonical QPSO implementation still shows materially higher QPSO cost on RC201 and C101. The evidence supports a discrete-encoding mismatch as a plausible explanation, not a proof of causality: random-key decoding turns a continuous QPSO move into a rank permutation, so arithmetic attraction in key space is only indirectly related to route quality.

Traffic in the current application is modeled or manually supplied. The ETA combines a conservative 25 km/h urban-speed floor, a configured delay multiplier, the OSRM street route’s generic duration, service time, and early-arrival waiting. No live traffic feed, GPS-based incident detection, or guaranteed real-time arrival prediction is connected.

2 Project purpose and system boundary

2.1 Operational problem

Given one depot and a set of customer deliveries, QMaps assigns each delivery to a vehicle, chooses a visit order for each vehicle, and schedules arrivals subject to vehicle capacity and customer time windows. The objective in the research benchmark is to reduce total route distance. In dispatch mode, travel-time estimates and optional priorities affect the schedule and can affect the optimizer’s soft cost.

The mathematical model below is a standard vehicle-flow formulation used to define the problem. The code does not call a mixed-integer programming solver. It searches over customer orders and applies a deterministic split decoder to create route sets.

2.2 Sets and parameters

Symbol	Meaning
$V = \{0\} \cup C$	Nodes: depot 0 and customer set $C = \{1, \dots, n\}$.
K	Available vehicles; vehicle k has capacity Q_k .

Symbol	Meaning
d_{ij}	Distance from i to j ; Euclidean in Solomon runs.
τ_{ij}	Travel time from i to j ; in benchmark files it follows the coordinate-time convention.
q_i	Demand at customer i ; depot demand is zero.
$[a_i, b_i]$	Allowed service-start time window for customer i .
s_i	Service duration at customer i .
M	A sufficiently large constant for conditional time constraints.
x_{ijk}	Binary variable: vehicle k travels directly from node i to j .
z_{ik}	Binary variable: vehicle k serves customer i .
T_{ik}	Service-start time at customer i when assigned to vehicle k .

3 Vehicle routing with time windows

3.1 Distance and cost

For Solomon coordinates (x_i, y_i) , the loader forms a complete distance matrix using unrounded Euclidean distance. The input coordinates are benchmark units, so the resulting total is a coordinate-distance cost, not automatically kilometers.

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (1)$$

Let K be the vehicle set and A the allowed directed arcs between distinct nodes. The standard distance objective is:

$$\min \sum_{k \in K, i \in V, j \in V} c_{ij} x_{ijk} \quad (2)$$

The benchmark evaluator sums every route's depot-to-customer, customer-to-customer, and customer-to-depot edges. The benchmark has no priority weights, so it reduces to this distance sum.

3.2 Service and assignment constraints

Every customer must be included exactly once. A vehicle that serves a customer must enter and leave that customer once, which links assignment to route flow.

$$\sum_{k \in K, j \in V \setminus \{i\}} x_{ijk} = 1 \quad \forall i \in C \quad (3)$$

$$\sum_{j \in V \setminus \{i\}} x_{jik} = \sum_{j \in V \setminus \{i\}} x_{ijk} = z_{ik} \quad (4)$$

For each vehicle, depot departure and depot return are balanced. A separate vehicle-use variable u_k can identify whether a route is active.

$$\sum_{j \in C} x_{0jk} = u_k = \sum_{i \in C} x_{i0k} \quad \forall k \in K \quad (5)$$

The total package demand assigned to a vehicle cannot exceed its capacity.

$$\sum_{i \in C} q_i z_{ik} \leq Q_k \quad \forall k \in K \quad (6)$$

The route, assignment, and vehicle-use decisions are binary; service-start times are nonnegative.

$$x_{ijk}, z_{ik}, u_k \in \{0,1\}; \quad T_{ik} \geq 0 \quad (7)$$

The flow equations need subtour elimination in a direct arc-based optimization model. QMaps' route decoder avoids detached customer cycles by emitting complete sequences that are split into depot-rooted routes.

3.3 Time-window constraints

Let T_{ik} denote the time service begins at i . If k travels from i to j , service at j cannot begin before travel and service at i have finished. Big-M relaxes the constraint when that arc is not selected.

$$T_{jk} \geq T_{ik} + s_i + \tau_{ij} - M(1 - x_{ijk}) \quad (8)$$

Time-window bounds apply to an assigned customer. The model allows waiting when a vehicle arrives before a window opens.

$$a_i - M(1 - z_{ik}) \leq T_{ik} \leq b_i + M(1 - z_{ik}) \quad (9)$$

$$A_i = D_{prev} + \tau_{prev,i}; \quad S_i = \max(A_i, a_i) \quad (10)$$

$$W_i = \max(0, a_i - A_i); \quad D_i = S_i + s_i \quad (11)$$

Feasibility requires service to start no later than b_i . In the current route checker, early arrivals are delayed to a_i , and the service-start time is checked against b_i . The return to the depot is checked against the depot's due time.

$$D_{last} + \tau_{last,0} \leq b_0 \quad (12)$$

3.4 Live priority cost

If the live request attaches priority weights, the route evaluator retains distance and adds a soft penalty proportional to each weighted service-start clock in hours. The default coefficient is 0.5. This component is absent from the Solomon benchmark runs.

$$J = D + \gamma \sum_{i \in C} w_i \frac{S_i}{60} \quad (13)$$

Priority is therefore a soft preference. It does not override capacity or a feasible time window, although the optimizer ranks candidate solutions by the returned numeric cost and stores feasibility separately.

4 Random keys and Prins split

4.1 Continuous position to customer order

PSO and QPSO operate on real-valued key vectors $x \in \mathbb{R}^n$. Sorting the keys produces a permutation π of customers. The decoder consumes rank only: adding a constant or applying a strictly increasing transformation leaves the order unchanged, except for ties.

$$\pi = \text{argsort}(x); \quad \pi_r \in \{1, \dots, n\} \quad (14)$$

ACO and QACO directly construct a sequence and convert its ranks back into keys. A* also produces an order that is converted to this common representation before route evaluation. The common decoder makes route evaluation consistent, but it does not make the optimizers' search moves equivalent.

4.2 Feasible route segments

For a permutation $\pi = (s_1, \dots, s_n)$, the split procedure considers contiguous subsequences. A segment from sequence position i through j represents one vehicle route: depot $\rightarrow s_i \rightarrow \dots \rightarrow s_j \rightarrow$ depot.

$$\ell_{ij} = d_{0,s_i} + \sum_{r=i}^{j-1} d_{s_r,s_{r+1}} + d_{s_j,0} \quad (15)$$

$$\sum_{r=i}^j q_{s_r} \leq Q \quad (16)$$

A segment is also time-window-feasible when the forward schedule, with early waiting, reaches every customer by its due time. The loader's split logic extends candidate segments incrementally and stops extending once capacity or a due-time constraint fails.

4.3 Dynamic-programming split

The unrestricted split computes the least distance over all feasible segmentations of the fixed sequence. F_j is the least cost for the first j sequence positions; each transition appends one feasible segment.

$$F_j = \min \{ F_i + \ell_{ij} : 0 \leq i < j \text{ and } \text{segment}(i,j) \text{ feasible} \} \quad (17)$$

When the live request provides a fleet limit, the implementation uses a route-count state $F_{k,j}$ and then selects the least-cost solution among feasible route counts up to that limit.

$$F_{k,j} = \min \{ F_{k-1,i} + \ell_{ij} : i < j \text{ and } \text{segment}(i,j) \text{ feasible} \} \quad (18)$$

The code tries time-window-feasible segments first. If no full split is found, it retries using capacity-feasible segments alone. In that fallback, the returned route can be capacity-feasible but fail time windows; the feasibility flag is checked by the evaluator. Saved full-suite CSV rows report all 30 runs feasible for every algorithm and instance.

The implementation minimizes distance over route counts subject to a fleet limit. It does not add a fixed cost per vehicle or lexicographically minimize vehicle count first. This distinction matters when comparing against benchmark solutions whose primary objective is fleet size.

5 Optimization algorithms

5.1 Classical particle swarm optimization

PSO stores a position x_i and velocity v_i for every particle. A particle is attracted toward its personal best p_i and the swarm's global best g . Independent random values r_1 and r_2 are sampled across particles and dimensions.

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (p_i - x_i) + c_2 r_2 (g - x_i) \quad (19)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (20)$$

Current PSO source defaults are 30 particles, inertia $\omega = 0.7$, and acceleration coefficients $c_1 = c_2 = 1.5$. Positions are clipped into $[0,1]$ before sorting. The bounded box does not constrain the route order to be valid or invalid; any finite key vector still decodes to a permutation.

5.2 Quantum-behaved particle swarm optimization

QPSO removes the velocity state. Each particle samples a local attractor between its personal best and the global best, while the population mean-best guides the scale of its logarithmic sampling step.

$$p_i = \varphi pbest_i + (1 - \varphi) gbest \quad (21)$$

$$mbest = \frac{\sum_{i=1}^N pbest_i}{N} \quad (22)$$

The current source uses a linearly contracting expansion coefficient from 1.0 to approximately 0.5 over the configured iteration count.

$$\beta(t) = 1 - 0.5 \frac{t}{T} \quad (23)$$

The QPSO position update is a direct stochastic sample around the local attractor. The sign is chosen randomly. There is no velocity update.

$$x_i(t+1) = p_i \pm \beta(t) |mbest - x_i(t)| \ln(1/u) \quad (24)$$

The implementation starts independent particles in $[0,1]$ for standalone QPSO. Keys are not clipped after the update; non-finite values are repaired before argsort. The constructor accepts `beta_initial` for compatibility and hybrid callers, but the current update computes β from `iteration/max_iter` and does not read `beta_initial`. Consequently, changing `beta_initial` alone does not alter the standalone QPSO contraction schedule.

5.3 Ant colony optimization

ACO constructs an order step by step. At node i , it chooses an unvisited customer j according to pheromone τ_{ij} and inverse-distance visibility η_{ij} . Probabilities are normalized over currently unvisited candidates.

$$\eta_{ij} = \frac{1}{d_{ij}} \quad (25)$$

$$P_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in U_k} \tau_{il}^\alpha \eta_{il}^\beta} \quad (26)$$

After an iteration, trails evaporate and each ant deposits an amount inversely proportional to its route cost on the directed edges in its constructed permutation.

$$\tau_{ij}(t+1) = (1 - \rho)\tau_{ij}(t) + \Delta\tau_{ij}^k; \quad \Delta\tau_{ij}^k = Q / L_k \quad (27)$$

Current ACO source defaults are 30 ants, $\alpha = 1$, $\beta = 3$, evaporation $\rho = 0.5$, and $Q = 1$. The project context note lists a different ACO profile (20 ants, $\beta = 5$, $\rho = 0.1$). The full-run CSV does not store a configuration snapshot, so the note and the saved historical values must not be treated as proof of which exact parameter profile generated each old file.

5.4 Quantum-inspired ant colony optimization

QACO stores one rotation angle θ_{ij} per directed edge. Its quantum-inspired state is represented classically by two amplitudes. The probability of observing state $|1\rangle$ is $\sin^2\theta$, and that probability is combined with the same inverse-distance heuristic used by ACO.

$$|\psi_{ij}\rangle = \cos(\theta_{ij})|0\rangle + \sin(\theta_{ij})|1\rangle \quad (28)$$

$$q_{ij} = \sin^2(\theta_{ij}); \quad P_{ij}^k \propto q_{ij}^\alpha \eta_{ij}^\beta \quad (29)$$

At each iteration, the current best ant has cost L_{best} . Other ants receive a nonnegative quality gap ΔE_k . The code uses a temperature floor of 0.05 and a capped rotation; edges in the iteration-best sequence rotate toward higher selection probability, while edges in another ant's sequence that are absent from the iteration best receive a weaker opposing rotation.

$$T(t) = \max(0.05, 1 - t/I); \quad \Delta E_k = \max(0, L_k - L_{best}) \quad (30)$$

$$\Delta\theta_k = \min(\pi/8, (\pi/2) \exp(-\alpha \frac{\Delta E_k}{T})) \quad (31)$$

$$\theta_{ij} \leftarrow \text{clip}(\theta_{ij} + \Delta\theta_k, 10^{-6}, \pi/2 - 10^{-6}) \quad (32)$$

Current QACO defaults are 20 ants, $\alpha = 1$, $\beta = 5$, and $\theta_0 = \pi/4$. This is the project's quantum-inspired rotation-angle implementation. The exact angle update is implementation-specific; do not claim an exact reproduction of a named paper without comparing its parameters and update rule. This remains a classical QACO variant, not execution on a quantum processor.

5.5 QA-QPSO hybrid

The hybrid first runs QACO for 40% of the configured iteration budget. It sends QACO's best key vector and cost to QPSO, which initializes one particle at that point and initializes the remaining particles by adding independent Gaussian noise with standard deviation 0.1. The QPSO phase receives a different seed.

$$IQACO = \max(1, \lfloor 0.4 I \rfloor); \quad IQPSO = I - IQACO \quad (33)$$

For more than 50 customers, the hybrid increases the QPSO population by 10. It also passes a larger $\beta_{initial}$ value, but the current QPSO update does not consume that parameter; the particle-count change does take effect. Runtime is divided using a 40% QACO allocation and the remaining measured time for QPSO.

The hybrid's empirical performance belongs to this exact staged implementation and budget. Its improvement cannot be attributed to QPSO in isolation because the hybrid includes QACO initialization and spends part of its total budget in each method.

5.6 A* search baseline

The code named A* searches bitmask states over customer orders under a time limit. It first evaluates deterministic and randomized candidate orders, then expands states using $f = g + h$. Completed orders are passed through the same split decoder.

The search-state path cost g is the sum of consecutive edges through the single customer sequence, whereas the reported final VRPTW objective is the split multi-route distance. Therefore the A* queue is not optimizing precisely the same cost that is reported after split. Treat these A* values as a project reference point, not as proof of an A* optimum or a directly equivalent metaheuristic comparison.

6 Dynamic traffic and realistic ETA

6.1 Static benchmark time and live dispatch time

The Solomon benchmark uses coordinates, demands, time windows, and service times. In the benchmark evaluator, distance and travel-time matrices follow the instance coordinate-time convention. The Navi Mumbai API builds a separate physical-distance matrix and initializes the travel-time matrix from an assumed 25 km/h urban speed. Scenario multipliers then alter selected pairwise links.

6.2 Congestion and incident adjustments

For an ordinary modeled delay δ_{ij} , the adjusted pairwise time is multiplied by $1 + \delta_{ij}$. The rush-hour scenario applies a 45% multiplier to the selected north-corridor pairs. The accident scenario applies a 2.2 multiplier, representing a 120% increase, on its selected pair. A manually entered closure assigns a large finite cost so the solver strongly avoids the pair while retaining a plan if no alternative exists.

$$\tau_{ij}' = \tau_{ij}(1 + \delta_{ij}); \quad \delta_{ij} \geq 0 \quad (34)$$

$$\tau_{ij}' = \tau_{ij} + P_{close} \quad (\text{large finite penalty in manual closure mode}) \quad (35)$$

The legacy research event simulator supports congestion and accident multipliers and can mark a closed matrix edge as infinite. The current manual Navi closure uses a finite penalty; it is therefore a strongly discouraged pairwise connection rather than a street-level hard exclusion. These are scenario models and do not come from live sensors.

6.3 Street geometry and ETA

After the optimizer chooses the visit sequence, the frontend can request road geometry and turn steps from the configured OSRM service. For each leg it compares the generic routing duration with a city-speed floor based on the road distance. It then applies the backend's configured traffic multiplier.

$$\tau_{ij}^{base} = 60 \frac{d_{ij}}{25} \quad \text{minutes, with distance in km} \quad (36)$$

$$m_{ij} = \max(1, \frac{\tau_{configured}}{\tau_{base}}); \quad \tau_{street} = \max(\tau_{OSRM}, 60 \frac{d_{road}}{25}) m_{ij} \quad (37)$$

The route-level ETA adds modeled street-driving time, service time, and early-arrival waiting. Stop arrival and departure clocks are propagated in sequence.

$$T_{route} = \sum_{legs} \tau_{leg} + \sum_{i \in C} s_i + W_i$$

(38)

This model avoids implausibly fast default profile estimates for short urban delivery legs, but 25 km/h is a configurable engineering assumption, not a calibration against a Navi Mumbai probe-vehicle dataset. The application's own terms describe travel-time and congestion as estimates and state that a live traffic feed is not connected.

6.4 What dynamic rerouting currently means

An incident or manual corridor edit changes the modeled pairwise travel-time matrix and the optimizer runs again. The selected visit order is then mapped to street geometry. This is useful for a controlled demonstration of route recovery, but it is not yet traffic sensing over an instrumented street graph. A pair of delivery locations is the unit of adjustment in the demo; a real street closure may affect only a subsegment of the roads between them.

A production-grade real-time system would need an approved live traffic provider or operational telemetry, time-dependent road-edge costs, incident confidence and expiry, GPS map matching, route-update delivery to drivers, and replayable event logs. Those capabilities are not claimed as currently implemented.

7 Software architecture and operating flow

The implementation separates user interaction, route optimization, and street presentation:

- Next.js frontend: collects locations, time windows, priorities, vehicle capacities, and modeled traffic settings; displays benchmarks, dispatch results, driver instructions, and the interactive route/city visual.
- FastAPI backend: validates request inputs, constructs distance and travel-time matrices, instantiates the selected optimizer, and returns route assignments, feasibility, costs, schedules, and optimization metadata.
- Optimizer layer: PSO/QPSO/ACO/QACO/QA-QPSO/A* evaluate customer orders through the shared decoder and split. The same evaluator reports cost, route list, convergence history, run time, iterations-to-best, and feasibility.
- Street routing layer: configured OSRM supplies road geometry, generic road duration, and turn-by-turn steps for the ordered stops. The frontend recalculates displayed distance and ETA when all route legs are mapped.

The product flow is: location and fleet inputs → route-order search → capacity/time-window split → feasible route assignments → street geometry and ETA decoration → dispatch and driver view. The visual city and moving vans are presentation of these route results; the solver itself works on a coordinate-derived complete graph and does not simulate a vehicle moving through a live traffic network.

7.1 Operational state and data boundary

The current dispatch demo stores shared state in API process memory. The project-readiness notes report that the driver selector is not authentication and the API does not enforce per-driver authorization. These are prototype boundaries, not optimization features. Before a real-company pilot, the deployment needs authenticated roles, server-side access control, persistent audited state if required, a retention/deletion policy, and production map/routing services.

8 Benchmark design and reproducibility

8.1 Data and run budget

The runner selects RC101 with 25 customers, RC201 with 50 customers, and C101 with 100 customers. The supplied instance files are read in customer-number order, and the runner takes the first n customers requested for each run. RC101 and RC201 are therefore reduced-size prefixes in this repository, not full 100-customer instances. The Solomon

family is a standard VRPTW benchmark family [1], but the selected prefixes and the project’s objective must be reported with the results.

Experiment component	Repository setting
Instances	RC101 prefix (25), RC201 prefix (50), C101 (100)
Algorithms	A*, PSO, QPSO, ACO, QACO, QA-QPSO
Runs	30 per algorithm and instance; seed = run index 0–29
Budget	At most 300 iterations or 5 seconds per standalone run
Saved fields	Cost, runtime, iterations to best, feasibility, convergence length and curve
Full-suite count	3 instances × 6 algorithms × 30 runs = 540 rows per CSV
Targeted QPSO rerun	RC201 and C101 × PSO/QPSO × 30 matched seeds = 120 rows

The main runner fixes each algorithm’s random seed to the run index. Its CSV contains repeated seed identifiers but does not record the exact constructor parameters, Git commit, Python/NumPy versions, or file hashes. Historical results are reproducible only to the extent that the source snapshot and environment used to produce them can be recovered.

8.2 Descriptive statistics

For R runs, the report uses arithmetic mean route cost and sample standard deviation. Lower cost is preferred. Feasibility is reported separately.

$$C = \frac{\sum_{r=1}^R C_r}{R} \quad (39)$$

$$s = \sqrt{\frac{\sum_{r=1}^R (C_r - C)^2}{R - 1}} \quad (40)$$

8.3 Pairwise and multi-algorithm tests

The saved analysis uses a Shapiro–Wilk check to inspect normality, Wilcoxon signed-rank tests for paired two-algorithm comparisons, and Friedman tests for six algorithms ranked across matched run blocks. This follows established nonparametric comparison practice for stochastic algorithms [8, 9]. A Nemenyi critical difference can be used for post-hoc average-rank comparisons.

$$W = \frac{(\sum_{i=1}^R a_i x_{(i)})^2}{\sum_{i=1}^R (x_{(i)} - \bar{x})^2} \quad (41)$$

$$d_r = C_{A,r} - C_{B,r} \quad (42)$$

$$W^+ = \sum_{r:d_r > 0} R_r ; \quad p = P(|W| \geq |W_{obs}| \mid H_0) \quad (43)$$

$$\chi^2 F = \frac{12N}{k(k+1)} \sum_{j=1}^k (R_j)^2 - 3N(k+1) \quad (44)$$

$$CD = q\alpha \sqrt{\frac{k(k+1)}{6N}} \quad (45)$$

The repository's signed-rank utility computes an exact two-sided rank-sum null probability for the paired differences. Zero differences are removed from the signed-rank calculation. The saved pairwise analysis reports pooled-standard-deviation Cohen's d:

$$s_p = \sqrt{\frac{(n_A - 1)s_A^2 + (n_B - 1)s_B^2}{n_A + n_B - 2}} \quad (46)$$

$$d = \frac{C_A - C_B}{s_p} \quad (47)$$

For a lower-cost comparison, negative d under the convention $d = (QPSO - PSO)/s_p$ favors QPSO; positive d favors PSO. In this report, the formula is written as the difference of the two means divided by the pooled standard deviation. For paired experiments, a paired effect size such as $d_z = \text{mean}(d_r)/\text{sd}(d_r)$ can also be reported, but it is not the effect-size definition stored in the project's current correction-run summary.

For paired data, a complementary standardized difference is $d_z = \text{mean}(d_r) / \text{sd}(d_r)$.

$$dz = \frac{\overline{d_r}}{s_d} \quad (48)$$

8.4 Multiplicity and scope

The archived per-instance Wilcoxon p-values and pairwise claims are raw and are not adjusted for the many algorithm-by-instance comparisons. The Friedman tests are per instance, not one pooled test over all three problem families. The targeted QPSO correction rerun covers only RC201 and C101 and is a separate experiment from the historical six-algorithm suite.

9 Results from saved experiments

9.1 Full six-algorithm suite

Table 1 uses the newer historical full-run file, results/benchmark_full_20260925_160939.csv. Each cell is mean $\pm$ sample standard deviation across 30 runs, in coordinate-distance cost units. Every saved row in that file is marked feasible. The two historical full-run CSV files are not identical; their means differ for several algorithms. They are kept separate and are not pooled.

Algorithm	RC101 25	RC201 50	C101 100
A*	901.24 $\pm$ 0.00	1737.87 $\pm$ 0.00	2378.58 $\pm$ 0.00
PSO	977.98 $\pm$ 43.80	2510.13 $\pm$ 65.16	4319.48 $\pm$ 77.82
QPSO	976.26 $\pm$ 36.21	2564.39 $\pm$ 67.08	4366.66 $\pm$ 49.91

Algorithm	RC101 25	RC201 50	C101 100
ACO	696.20 ± 32.82	1447.93 ± 43.11	1824.99 ± 152.43
QACO	661.11 ± 23.62	1406.17 ± 35.31	1987.36 ± 115.90
QA-QPSO	557.54 ± 29.84	1233.69 ± 54.72	1897.01 ± 121.47

The A* rows have zero sample standard deviation in this file. More importantly, the A* implementation's search-state path cost and the split route objective are not identical, so A* is not a like-for-like benchmark comparator. Do not interpret a lower or higher A* table value as an optimality result.

9.2 Historical PSO and QPSO comparison

The saved statistical summary for the newer historical CSV reports the following paired comparisons. P-values are unadjusted; d is pooled-standard-deviation Cohen's d using QPSO minus PSO.

Instance	PSO mean	QPSO mean	Wilcoxon p	Cohen d	Interpretation
RC101	977.98	976.26	0.7922	-0.043	No detectable difference; negligible effect
RC201	2510.13	2564.39	0.0006	+0.821	PSO lower cost; large effect
C101	4319.48	4366.66	0.0234	+0.722	PSO lower cost; medium effect

The per-instance Friedman omnibus tests in the saved output reject equal algorithm ranks on all three cases (RC101 $p = 2.85 \times 10^{-29}$, RC201 $p = 2.69 \times 10^{-29}$, C101 $p = 7.11 \times 10^{-28}$). This only establishes that at least some algorithms differ on each instance; it does not make every pairwise comparison significant.

9.3 Matched QPSO correction rerun

The separate file results/qpso_correction_20260928_002809.csv labels QPSO implementation canonical-qpso-unbounded-keys-v2. It contains 30 matched seeds for PSO and QPSO on RC201 and C101 under a 300-iteration/5-second budget. The project's exact paired Wilcoxon utility was applied to these saved costs for this report.

Instance	PSO mean	QPSO mean	QPSO cost excess	Wilcoxon p	Cohen d
RC201	1896.38	2424.36	+27.84%	6.15×10^{-8}	+2.75
C101	3357.94	4405.49	+31.20%	1.86×10^{-9}	+7.77

All 120 rows are marked feasible. The unadjusted p-values remain below 0.05 after a two-comparison Bonferroni threshold of 0.025. This targeted rerun is strong evidence that QPSO's current corrected implementation still performs worse than PSO on these two saved cases; it does not establish performance on RC101 or on other instances.

The corrected rerun's PSO means differ substantially from the historical suite. The studies are separate snapshots and should not be merged or compared as if only QPSO changed. A future publication-grade rerun should pin one source commit, record all algorithm parameters and data hashes, and execute every algorithm from that same snapshot.

9.4 Reading the results responsibly

The full-suite table supports a narrow statement: on the archived selected instances and the project's cost definition, the hybrid had the lowest mean on RC101 and RC201, while ACO had the lowest mean on C101. It does not support a universal statement that the hybrid or any quantum-inspired method always wins. Benchmark best-known comparisons require the same customer set, capacity/fleet conditions, distance precision, and objective ordering.

SINTEF's Solomon VRPTW reference uses a hierarchical objective: first minimize vehicle count, then minimize total distance [10]. QMaps' current split minimizes distance among routes within the available fleet bound; it does not enforce the same lexicographic objective. Therefore the values in Table 1 should be described as internal comparative costs, not as Solomon best-known gaps.

10 Why QPSO underperforms on larger instances

10.1 What the data says

The historical 30-run comparison shows QPSO statistically indistinguishable from PSO on the 25-customer RC101 prefix, then worse mean cost on the 50-customer RC201 prefix and 100-customer C101 instance. The corrected matched rerun repeats the direction on the two larger instances. This pattern is consistent with a representation and scaling problem, but a benchmark comparison alone cannot identify the cause.

10.2 Continuous attraction meets a permutation decoder

Random-key encoding maps an n -dimensional continuous point to a discrete order only through pairwise key rankings. The decoded solution changes when keys cross. Within a region where all pairwise rankings stay fixed, many different key vectors represent the same permutation. The objective seen by QPSO is consequently piecewise constant or discontinuous across ranking boundaries rather than a smooth function of Euclidean key-space distance.

QPSO computes local attractors and mbest using arithmetic averages of numeric keys. Yet two vectors that are numerically close can encode different meaningful adjacency patterns, and two distant vectors can encode the same order. Arithmetic averaging in key coordinates therefore does not directly average routes, customer adjacencies, or split boundaries.

10.3 Why the gap can grow with n

- The number of key coordinates grows with customer count, increasing the chance that a sampled update changes many relative ranks at once.
- A small numeric move can leave every key ordering unchanged, so the route evaluator returns the same discrete plan even though the particle moved.
- A large move can reorder many customers together, producing a broad route change that is difficult to refine with the continuous attractor.
- The Prins split is deterministic for a given order. It can optimize segment boundaries for that order, but QPSO changes vehicle composition only indirectly by changing the permutation.
- The canonical QPSO contraction parameter is scheduled by iteration, but the `beta_initial` argument is currently unused. Hybrid code that passes a larger `beta_initial` for $n > 50$ does not actually increase QPSO's exploration radius.

10.4 What was corrected and what remains

The current QPSO code uses the no-velocity quantum-behaved update, independent initial samples, an updated global best, and unbounded finite random keys. These are meaningful implementation corrections. The rerun demonstrates that correcting those mechanics does not by itself solve the mismatch between continuous key-space dynamics and a rank-decoded routing objective.

The evidence does not show that QPSO is intrinsically inferior to PSO. It shows that this QPSO configuration plus this encoding/decoder and short run budget is inferior on the two tested larger cases. A well-designed permutation-aware QPSO hybrid could behave differently and needs a separately registered experiment.

11 Evidence limits and recommended next experiments

11.1 Limitations to state to judges

- No live traffic provider, real-time incident detector, or live vehicle GPS stream is connected. Current traffic is selected from scenarios or entered as a modeled adjustment.
- Optimizer edges are based on coordinate-derived pairwise distances and times. OSRM street geometry is attached after stop order selection; the metaheuristic does not optimize directly on a time-dependent road graph.
- The 25 km/h urban-speed floor is a transparent model parameter, not empirically calibrated or a probabilistic ETA interval.
- RC101 and RC201 are reduced-size prefixes in the benchmark runner. Their results are not identical to runs on the full canonical 100-customer files.
- The objective does not use the standard Solomon/SINTEF lexicographic fleet-count-first objective, so internal mean costs are not best-known-solution gaps.
- Historical CSV files do not include code commit, complete parameter snapshot, library versions, or data hashes. Two full-suite CSV snapshots differ.
- The A* path-expansion cost and post-split route-distance cost differ, limiting direct comparison.
- The saved benchmark comparison uses raw pairwise p-values without an across-all-comparisons multiplicity adjustment. The report labels them unadjusted.
- The matched QPSO correction rerun covers only RC201 and C101. No corrected 30-seed RC101 rerun is present.

11.2 Recommended experimental sequence

Step	Action	Reason
1	Pin a Git commit, environment versions, data checksums, and constructor parameters in every CSV row or a linked manifest.	Makes each benchmark result auditable and repeatable.
2	Align all methods to the same Solomon customer set and objective; report fleet count first, then distance, or explicitly define a different objective.	Prevents a method from appearing better by using extra vehicles or a different cost.
3	Run all six algorithms on all three instances with the same 30 seeds and the same wall-clock/iteration policy.	Keeps the comparison paired and complete.
4	Add a permutation-aware QPSO neighborhood: swap/insert moves, rank-space distance, or an order-preserving map; preserve the canonical QPSO baseline separately.	Tests the discrete-encoding hypothesis without changing the baseline retrospectively.
5	If beta_initial is intended to matter, wire it into a documented schedule, then compare with the current fixed schedule as a separate ablation.	Resolves the unused-parameter mismatch.
6	Record best, mean, median, standard deviation, feasibility, vehicle count, wall time, convergence curves, paired Wilcoxon, an effect size suited to paired data, and multiplicity-adjusted post-hoc results.	Shows quality, reliability, runtime, and uncertainty.
7	Calibrate ETA against observed trips and compare predicted versus actual travel times by road class and time of day.	Turns the 25 km/h floor from an assumption into a measured model.
8	Model live road segments with provider-supplied or telemetry-derived time-dependent edge costs, then replay incidents with confidence and expiry.	Moves dynamic rerouting toward an operational road-network model.

11.3 Recommended judge-facing claim

A defensible summary is: “QMaps combines a VRPTW route-order search with capacity/time-window splitting, a dispatch interface, and scenario-based congestion rerouting. In the saved 30-run internal suite, the QA-QPSO hybrid has the lowest mean cost on the RC101 and RC201 subsets, while ACO is lowest on C101. QPSO alone underperforms PSO on the larger tested cases, including a separate corrected 30-seed rerun. Traffic inputs and ETAs are modeled rather than live.”

Avoid saying that a quantum method universally outperforms classical methods, that benchmark values match published best-known Solomon solutions, or that QMaps detects real-time accidents. Those claims are not supported by the inspected implementation and saved artifacts.

12 Glossary

Term	Definition in this report
VRP	Vehicle Routing Problem: assigning customers to routes and ordering visits.
VRPTW	Vehicle Routing Problem with Time Windows: service at each customer must begin within an allowed interval.
Random key	A real number associated with each customer; argsort yields a customer permutation.
Prins split	Dynamic-programming procedure that partitions a fixed customer order into routes.
Personal best	Best decoded solution previously found by one PSO/QPSO particle.
Global best	Best decoded solution found by the swarm so far.
mbest	Coordinate-wise mean of all particles’ personal-best vectors in QPSO.
Feasible	Plan satisfies the capacity/time-window checks used by the current evaluator.
Modeled congestion	A user- or scenario-supplied multiplier applied to a pairwise travel-time estimate.
OSRM	Open Source Routing Machine; configured service used for generic road routes, geometry, and maneuver steps.
Cohen’s d	Standardized mean difference; the project currently uses a pooled-standard-deviation denominator.
Matched seed	A run index reused for each algorithm so results can be paired by experimental block.

13 References

- [1] Solomon, M. M. (1987). Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. *Operations Research*, 35(2), 254–265. doi:10.1287/opre.35.2.254. [INFORMS record](#)
- [2] Bean, J. C. (1994). Genetic Algorithms and Random Keys for Sequencing and Optimization. *ORSA Journal on Computing*, 6(2), 154–160. doi:10.1287/ijoc.6.2.154. [INFORMS record](#)
- [3] Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12), 1985–2002. doi:10.1016/S0305-0548(03)00158-8. [Publisher DOI](#)
- [4] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of the IEEE International Conference on Neural Networks*, 4, 1942–1948. doi:10.1109/ICNN.1995.488968. [IEEE DOI](#)
- [5] Sun, J., Xu, W., & Feng, B. (2004). A global search strategy of quantum-behaved particle swarm optimization. *IEEE Conference on Cybernetics and Intelligent Systems*, 111–116. doi:10.1109/ICCIS.2004.1460396. [IEEE record](#)

- [6] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. doi:10.1109/TSSC.1968.300136. [IEEE DOI](#)
- [7] Dorigo, M., Maniezzo, V., & Coloni, A. (1996). Ant System: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1), 29–41. [Publisher record](#)
- [8] Demšar, J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7, 1–30. [JMLR article](#)
- [9] Derrac, J., García, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, 1(1), 3–18. doi:10.1016/j.swevo.2011.02.002. [Publisher DOI](#)
- [10] SINTEF Applied Mathematics. Solomon 100-customer VRPTW instances and best-known solutions. The page states a hierarchical objective of first minimizing vehicles and then distance. [Benchmark reference](#)

Appendix A Current implementation parameters

This table lists current class defaults inspected in the repository. It does not reconstruct the exact parameters used in historical CSV snapshots because those files do not serialize complete configurations.

Method	Current source defaults and behavior
PSO	30 particles; $\omega = 0.7$; $c_1 = 1.5$; $c_2 = 1.5$; keys clipped to $[0,1]$.
QPSO	30 particles; $\beta(t) = 1 - 0.5t/T$; no velocity; finite keys remain unbounded; beta_initial is stored but unused in the update.
ACO	30 ants; $\alpha = 1$; $\beta = 3$; $\rho = 0.5$; $Q = 1$.
QACO	20 ants; $\alpha = 1$; $\beta = 5$; $\theta_0 = \pi/4$; probability is $\sin^2\theta$ times distance heuristic.
QA-QPSO	QACO uses 40% of configured iterations; QPSO uses remainder; QPSO population increases by 10 for $n > 50$; warm start around QACO best.
A*	Bitmask customer-order search with a 5-second default time limit; queue-state path cost does not equal post-split route cost.

Parameter differences between current defaults and AGENTS.md project notes, particularly ACO, should be reconciled before the next formal run. The report favors code evidence over stale intended configuration.

Appendix B Mathematical symbols

Symbol	Meaning
i, j, k	Node, node, and vehicle indices.
C, V, K	Customer set, all nodes including depot, and available vehicles.
d_{ij}, τ_{ij}	Distance and travel time from i to j .
q_i, Q_k	Customer demand and vehicle capacity.
a_i, b_i, s_i	Opening time, closing time, and service duration.
π, s_r	Customer permutation and customer at sequence position r .
x_{ijk}, z_{ik}, u_k	Arc choice, customer-to-vehicle assignment, and vehicle-use indicator.

Symbol	Meaning
$T_{ik}, A_i, S_i, W_i, D_i$	Service time, arrival time, service start, waiting duration, and departure time.
$pbest_i, gbest, mbest$	Particle best, swarm best, and mean best position.
$\alpha, \beta, \rho, \theta$	ACO/QACO transition exponents, evaporation rate, and QACO rotation angle; QPSO β is contraction-expansion.
R, k, N	Number of repeated runs/blocks, number of algorithms, and sample/block count in a statistical test.

End of report.